



Module Objective: In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

Before you start: This lab continues the project from Labs 1–4. Open the same project and check that `SearchAgent.astar_search()` in `agent.py` works.

- Parts 1, 2 & 4: new files (`logic_engine.py` , `test_logic.py`) in the project folder.
- Part 3: edit your existing `agent.py` .
- **Create a separate branch `lab-07`** and start your work there. When finished, **commit your code and push the branch**, as you did in Labs 1–4.
- **Upload this worksheet's answered PDF to the submission portal.**

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .
2. Implement a `KnowledgeBase` class based on the following structural requirements:

```

CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set                // To store unique string facts (e.g.,
    "TargetVisible")
    DECLARE rules AS List               // To store rules as Tuples: (
    [list_of_premises], "conclusion_string" )

    // Methods
    FUNCTION tell_fact(fact_string):
        ADD fact_string TO facts

    FUNCTION tell_rule(premise_list, conclusion_string):
        APPEND Tuple(premise_list, conclusion_string) TO rules

    FUNCTION clear_facts():
        EMPTY the facts Set

```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```

FUNCTION forward_chain():
    DECLARE new_facts_added = TRUE

    WHILE new_facts_added == TRUE:
        new_facts_added = FALSE

        FOR EACH (premises, conclusion) IN rules:
            IF conclusion IS NOT IN facts:

                // Modus Ponens Check
                IF ALL premises ARE IN facts:
                    ADD conclusion TO facts
                    new_facts_added = TRUE

```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In `agent.py`, add `from logic_engine import KnowledgeBase`, then create the KB in `SearchAgent.__init__`: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
 - Rule 1:* `TargetVisible  $\wedge$  HasDust  $\Rightarrow$  SafeToEngage`
 - Rule 2:* `SafeToEngage  $\wedge$  BloodseekerMissing  $\Rightarrow$  Retreat`

The game does not produce these facts, so you supply them per tile as a dictionary: `{(x, y): [facts]}`. A tile where `Retreat` is deduced is unsafe to enter.

Step 3.2: Validating Feasibility in A*

Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Add a parameter `tile_facts=None` to `astar_search()` (treat `None` as `{}`).
2. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).

3. Before adding a neighbor to the `open_list` , clear the KB facts.
4. Feed that tile's facts into the KB: `for fact in tile_facts.get(successor, []): self.kb.tell_fact(fact) .`
5. Run `self.kb.forward_chain()` .
6. If `'Retreat'` is deduced and exists in `self.kb.facts` , mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Step 3.3: Run It in the Grid

1. In `sense_and_act()` , define sample facts and pass them to A*:

```
tile_facts = {
    (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissing'], # deduces
    Retreat: blocked
    (0, 1): ['TargetVisible', 'HasDust'], # SafeToEngage
    only: allowed
}
self.plan = self.astar_search(position, nearest_food, walls, grid_size,
tile_facts=tile_facts)
```

2. Run `visual_grid_game.py` . The agent should never step onto `(1, 0)` .

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py` . Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```

from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEngage')
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], 'Retreat')

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Should deduce SafeToEngage"
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Should NOT deduce Retreat"

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should deduce Retreat to override search"

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()

```

Part 5: Theory-to-Implementation Mapping

Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)

In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

Reachable is a physical check. My `_neighbors()` helper only yields cells that are inside the grid and not in `walls`. If a cell fails this test, the agent cannot physically move there.

Feasible is a logical check, applied after a cell has passed the reachability test. Before a neighbour is pushed onto the A* frontier, the code:

1. calls `self.kb.clear_facts()`
2. tells the KB that tile's percepts from `tile_facts`
3. runs `self.kb.forward_chain()`
4. if `'Retreat'` is in `self.kb.facts`, uses `continue` so the tile is never added to the open list

So a tile must be both reachable and not provably unsafe. Tile (1,0) is open floor, but its facts (TargetVisible, HasDust, BloodseekerMissing) let the KB derive SafeToEngage and then Retreat, so it gets pruned. A* plans (0,0)→(2,0) as Up, Right, Right, Down instead of Right, Right. Tile (0,1) only derives SafeToEngage, so it stays feasible.

The KB enforces feasibility by acting as a filter on the successor function: A* can only expand states it cannot prove lead to Retreat. Clearing the facts for each tile keeps one tile's facts from affecting another.

2. Declarative vs. Procedural Paradigms (Analyze)

Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

The KB separates **what** the agent knows (rules, stored as data) from **how** it reasons (one generic `forward_chain()` engine).

A hard-coded `if 'TargetVisible' in percepts and 'HasDust' in percepts:` mixes domain knowledge into the movement code. Rule 2 depends on the result of Rule 1, so the procedural version needs nested or carefully ordered ifs. Each new rule means editing, re-ordering and retesting A* itself.

In the declarative design, A* only asks one question: is 'Retreat' in `kb.facts` ? Chaining is automatic: the while-loop repeats until it reaches a fixed point, so rules can be added in any order and multi-step deductions still happen.

With 50 new rules:

- **Procedural:** the movement code grows into a large tangle of branches. The number of possible interactions grows quickly, the code is hard to read and verify, and bugs are likely.
- **Declarative:** we add 50 `tell_rule(...)` lines. The agent and A* code stay the same size, and the rules can be tested in isolation, as `test_logic.py` does, or even loaded from a file.

The trade-off is runtime cost. Each pass is $O(\text{rules} \times \text{premises})$, repeated up to the number of derivable facts, and it runs for every successor. That cost is small for this game.

3. Modus Ponens & Horn Clauses (Understand)

The rule `['TargetVisible', 'HasDust']  $\Rightarrow$  'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

A **Horn clause** is a disjunction with at most one positive literal. A definite clause like `TargetVisible  $\wedge$  HasDust  $\Rightarrow$  SafeToEngage` is equivalent to `$\neg$ TargetVisible  $\vee$   $\neg$ HasDust  $\vee$  SafeToEngage`.

Modus Ponens says: from `$P_1 \wedge \dots \wedge P_n \Rightarrow Q$` and the facts `$P_1, \dots, P_n$` , infer `$Q$` .

My `forward_chain()` carries out this rule directly:

- Each tuple `(premises, conclusion)` is the implication.
- `all(p in self.facts for p in premises)` checks that every antecedent has been established.
- `self.facts.add(conclusion)` asserts `Q`.

The `while new_facts_added` loop applies Modus Ponens repeatedly. A conclusion from one rule, such as `SafeToEngage`, becomes a premise for the next, such as `Retreat`. The loop stops when a full pass adds nothing new (a fixed point).

Because the KB contains only Horn clauses, this data-driven forward chaining is **sound**: it only derives facts that are entailed. It is also **complete** for atomic facts: it derives every entailed one. It always terminates, since there are finitely many symbols and the `conclusion not in facts` check stops any fact from being added twice.

